Day 35: Veda Technology Business & Service Analytics

Notebook PDF Export • Data Science Track

Important: This project uses synthetic data only. Findings are for practice and must not be treated as actual Veda Technology business results.

#

Code

```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)
sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", 50)
```

1. Generate a synthetic dataset

```
n = 1200
dates = pd.to_datetime(rng.choice(pd.date_range("2026-01-01", "2026-09-30", freq="D"), size=n))
areas = ["Internships", "Training", "IT Services", "Digital Services"]
channels = ["Organic Search", "Social Media", "Referral", "Paid Ads", "Direct"]
services = ["Website Development", "Data Analytics", "AI Solutions", "Cloud Support", "General Inquiry"]
cities = ["Delhi NCR", "Mumbai", "Pune", "Bengaluru", "Hyderabad", "Other"]

area = rng.choice(areas, n, p=[.28, .27, .24, .21])
channel = rng.choice(channels, n, p=[.30, .24, .15, .16, .15])
service = rng.choice(services, n)
city = rng.choice(cities, n)
visitors = rng.poisson(110, n) + np.where(channel == "Paid Ads", 25, 0)
inquiries = rng.poisson(12, n) + np.where(area == "IT Services", 4, 0)
applications = np.where(area == "Internships", rng.poisson(18, n), rng.poisson(3, n))
enrollments = np.where(area == "Training", rng.poisson(22, n), rng.poisson(2, n))
response_hours = np.clip(rng.gamma(2.1, 5.0, n), .5, 72)
satisfaction = np.clip(np.round(5 - response_hours / 24 + rng.normal(0, .55, n)), 1, 5).astype(int)
demand = inquiries + applications + enrollments
marketing_spend = np.where(channel == "Paid Ads", rng.uniform(500, 5000, n), rng.uniform(0, 900, n))
revenue_proxy = (np.where(area == "IT Services", 2200, 0) +
                 np.where(area == "Digital Services", 1700, 0) +
                 np.where(area == "Training", 800, 0) +
                 np.where(area == "Internships", 150, 0) +
                 demand * rng.uniform(25, 90, n) + rng.normal(0, 250, n))
df = pd.DataFrame({
    "date": dates, "business_area": area, "marketing_channel": channel,
    "service_type": service, "city": city, "website_visitors": visitors,
    "inquiries": inquiries, "internship_applications": applications,
    "training_enrollments": enrollments, "response_hours": response_hours.round(2),
    "satisfaction_score": satisfaction, "marketing_spend": marketing_spend.round(2),
    "service_demand": demand, "revenue_proxy": np.maximum(revenue_proxy, 0).round(2)
})
# Introduce a few missing values and duplicate records for cleaning practice.
for col in ["city", "service_type", "response_hours"]:
    df.loc[rng.choice(df.index, 12, replace=False), col] = np.nan
df = pd.concat([df, df.sample(8, random_state=RANDOM_STATE)], ignore_index=True)
print("Dataset shape:", df.shape)
display(df.head())
display(df.describe(include="all").T)
```

2. Data quality checks and cleaning

```
print("Missing values before cleaning:")
display(df.isna().sum().sort_values(ascending=False).to_frame("missing_count"))
print("Duplicate rows:", int(df.duplicated().sum()))
clean_df = df.drop_duplicates().copy()
clean_df["date"] = pd.to_datetime(clean_df["date"], errors="coerce")
for col in clean_df.select_dtypes(include=np.number).columns:
    clean_df[col] = clean_df[col].fillna(clean_df[col].median())
for col in clean_df.select_dtypes(include="object").columns:
    if clean_df[col].isna().any():
        clean_df[col] = clean_df[col].fillna(clean_df[col].mode().iloc[0])
clean_df = clean_df.dropna(subset=["date"])
print("Cleaned shape:", clean_df.shape)
print("Remaining missing values:", int(clean_df.isna().sum().sum()))
display(clean_df.head())
```

3. KPI summary

```
total_visitors = clean_df["website_visitors"].sum()
total_demand = clean_df["service_demand"].sum()
kpis = pd.DataFrame({
```

```
    "KPI": ["Website visitors", "Inquiries", "Internship applications", "Training enrollments",
            "Total demand events", "Average response time (hours)", "Average satisfaction (1–5)",
            "Marketing spend (synthetic)", "Revenue proxy (synthetic)"],
    "Value": [total_visitors, clean_df.inquiries.sum(), clean_df.internship_applications.sum(),
              clean_df.training_enrollments.sum(), total_demand, clean_df.response_hours.mean(),
              clean_df.satisfaction_score.mean(), clean_df.marketing_spend.sum(),
              clean_df.revenue_proxy.sum()]
})
display(kpis.round(2))
```

4. Business-area performance

```
area_summary = clean_df.groupby("business_area").agg(
    records=("business_area", "size"), visitors=("website_visitors", "sum"),
    inquiries=("inquiries", "sum"), applications=("internship_applications", "sum"),
    enrollments=("training_enrollments", "sum"), avg_response_hours=("response_hours", "mean"),
    avg_satisfaction=("satisfaction_score", "mean"), revenue_proxy=("revenue_proxy", "sum")
).sort_values("revenue_proxy", ascending=False)
display(area_summary.round(2))
sns.barplot(data=area_summary.reset_index(), x="business_area", y="revenue_proxy")
plt.title("Synthetic Revenue Proxy by Business Area")
plt.xticks(rotation=20, ha="right")
plt.tight_layout()
plt.show()
```

5. Monthly visitor and demand trends

```
monthly = clean_df.assign(month=clean_df.date.dt.to_period("M").astype(str)).groupby("month").agg(
    visitors=("website_visitors", "sum"), inquiries=("inquiries", "sum"),
    applications=("internship_applications", "sum"), enrollments=("training_enrollments", "sum")
).reset_index()
display(monthly)
monthly.set_index("month")[["visitors", "inquiries", "applications", "enrollments"]].plot(
    figsize=(11, 5), marker="o")
plt.title("Monthly Website Traffic and Demand")
plt.xticks(rotation=35, ha="right")
plt.tight_layout()
plt.show()
```

6. Marketing-channel analysis

```
channel_summary = clean_df.groupby("marketing_channel").agg(
    visitors=("website_visitors", "sum"), demand=("service_demand", "sum"),
    spend=("marketing_spend", "sum"), satisfaction=("satisfaction_score", "mean")
)
channel_summary["demand_per_visitor"] = channel_summary.demand / channel_summary.visitors.replace(0, np.nan)
channel_summary["spend_per_demand_event"] = channel_summary.spend / channel_summary.demand.replace(0, np.nan)
display(channel_summary.sort_values("demand_per_visitor", ascending=False).round(3))
sns.barplot(data=channel_summary.reset_index(), x="marketing_channel", y="demand_per_visitor")
plt.title("Demand Events per Visitor by Marketing Channel")
plt.xticks(rotation=20, ha="right")
plt.tight_layout()
plt.show()
```

7. Service demand and segmentation

```
service_summary = clean_df.groupby("service_type").agg(
    total_demand=("service_demand", "sum"), avg_response_hours=("response_hours", "mean"),
    avg_satisfaction=("satisfaction_score", "mean"), revenue_proxy=("revenue_proxy", "sum")
).sort_values("total_demand", ascending=False)
display(service_summary.round(2))
segment_summary = clean_df.groupby(["city", "business_area"]).agg(
    records=("business_area", "size"), demand=("service_demand", "sum"),
    avg_satisfaction=("satisfaction_score", "mean")
).sort_values("demand", ascending=False)
display(segment_summary.head(12).round(2))
sns.barplot(data=service_summary.reset_index(), x="service_type", y="total_demand")
plt.title("Total Demand by Service Type")
plt.xticks(rotation=25, ha="right")
plt.tight_layout()
plt.show()
```

8. Correlation analysis

```
corr_cols = ["website_visitors", "inquiries", "internship_applications", "training_enrollments",
             "response_hours", "satisfaction_score", "marketing_spend", "service_demand", "revenue_proxy"]
corr = clean_df[corr_cols].corr(numeric_only=True)
plt.figure(figsize=(10, 7))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Correlation Matrix of Numeric Business Metrics")
plt.tight_layout()
plt.show()
display(corr.round(2))
```

9. Predictive analysis: synthetic revenue proxy

```
features = ["business_area", "marketing_channel", "service_type", "city",
            "website_visitors", "inquiries", "internship_applications",
            "training_enrollments", "response_hours", "satisfaction_score", "marketing_spend"]
X, y = clean_df[features], clean_df["revenue_proxy"]
cat_cols = ["business_area", "marketing_channel", "service_type", "city"]
num_cols = [c for c in features if c not in cat_cols]
preprocessor = ColumnTransformer([
    ("num", Pipeline([("imputer", SimpleImputer(strategy="median")), ("scale", StandardScaler())]), num_cols),
    ("cat", Pipeline([("imputer", SimpleImputer(strategy="most_frequent")),
                      ("onehot", OneHotEncoder(handle_unknown="ignore"))]), cat_cols)
])
model = Pipeline([
```

```
    ("preprocessor", preprocessor),
    ("regressor", RandomForestRegressor(n_estimators=150, random_state=RANDOM_STATE, min_samples_leaf=3))
])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.2, random_state=RANDOM_STATE)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(f"MAE:  {mean_absolute_error(y_test, pred):,.2f}")
print(f"RMSE: {mean_squared_error(y_test, pred) ** 0.5:,.2f}")
print(f"R²:   {r2_score(y_test, pred):.3f}")
display(pd.DataFrame({"actual": y_test, "predicted": pred}).head(10))
```

#